

Chapter 2

DNA Strings: Indexing, len(), and Useful Methods

Treat a DNA sequence as text, then learn how to inspect and transform it.

EDOS learning idea

We will use biology and DNA examples because they are concrete, visual, and easy to experiment with. Type the examples yourself, change one thing, run again, and observe what changed.

What you will be able to do

- Use len() with DNA strings.
- Read individual bases using indexes.
- Use slicing to extract a region.
- Use string methods such as count(), upper(), lower(), replace(), and find().
- Calculate a simple GC percentage.

Classroom rhythm

Read a small idea -> run the example -> change one value -> predict the result -> run again -> explain what happened in your own words.

2.1 DNA is a Python string

A DNA sequence fits naturally into a Python string. Once it is stored as a string, you get many useful operations for free.

```
dna = 'ATGCGTACGTTAGC'
print(dna)
print(type(dna))
```

`type()` tells you what kind of object Python is holding. For `dna`, the result is `str`, which means string.

2.2 `len()`: how long is the sequence?

`len(dna)` counts the number of characters. Because each character represents one base, this is also the number of bases in the sequence.

```
dna = 'ATGCGTACGTTAGC'
print(len(dna))
```

2.3 Indexing: pick one base

Python starts counting positions from 0, not 1. The first character is `dna[0]`, the second is `dna[1]`, and so on.

```
dna = 'ATGCGT'
print(dna[0])    # A
print(dna[1])    # T
print(dna[-1])   # last base
```

Negative indexes count from the end. `dna[-1]` is the last character.

2.4 Slicing: take a piece of DNA

A slice extracts a range. `dna[start:stop]` includes start but stops before stop.

```
dna = 'ATGCGTACGTTAGC'
first_codon = dna[0:3]
middle = dna[3:9]
print(first_codon)
print(middle)
```

This start-included, stop-excluded rule is very common in Python.

2.5 String methods you will actually use

```
dna = 'atgcgtaacgtt'
print(dna.upper())
print(dna.count('a'))
print(dna.replace('a', 'A'))
print(dna.find('cg'))
```

A method is a function attached to an object. The dot in `dna.upper()` means: ask this string to perform its upper operation.

Useful methods: `count()` counts matches; `upper()` and `lower()` change letter case; `replace()` substitutes text; `find()` returns the first position of a pattern, or `-1` if it is missing.

2.6 A simple GC percentage

GC percentage is the fraction of G and C bases in a DNA sequence. We can calculate it with `count()` and `len()`.

```
dna = 'ATGCGCGTAA'
gc_count = dna.count('G') + dna.count('C')
gc_percent = gc_count / len(dna) * 100
print('GC %:', gc_percent)
```

Later we will turn this calculation into a reusable function.

Try it yourself

These are practice prompts, not a test. If your code breaks, that is useful information. Read the error, change one thing, and try again.

1. Print the first and last base of a DNA sequence.
2. Extract the first 3 bases as a codon.
3. Count A, C, G, and T using `count()`.
4. Convert a lowercase sequence to uppercase.
5. Calculate GC percentage for ATATGCGCGC.

Tiny debugging habit

When something fails, check spelling, quotation marks, brackets, colons, and indentation first. Then print intermediate values so you can see what Python sees.

Chapter summary

- Strings are ideal for DNA sequences.
- Python indexes start at 0.
- Slicing extracts regions.
- String methods save a lot of manual work.
- GC percentage combines `count()` with `len()`.

Next step

Open Notebook 02 in EDOS Jupyter and run the matching exercises. Then continue to Chapter 3.

EDOS - Educational Desktop Operating Simulator
Copyright by Dr. Yash Munnalal Gupta